SC2002 Restaurant Rush Group Assignment

Java command-line turn-based restaurant management game | Student brief

Your group builds a playable restaurant management game in Java. The player is the manager. Staff time is limited, so the manager decides which customers and orders come first. You build the game in three stages, and each stage must show how your object-oriented design supports the next set of requirements.

Item	Summary
What you build	A turn-based restaurant game with a command-line interface, in three runnable stages.
Stages	All three stages are published now and submitted together. Each stage is a tagged, runnable version in GitHub.
Interface	A command-line interface is enough. A GUI is optional and earns no extra marks by itself.
Submission	One zip per group: source code, README, PDF report, recorded video and declarations (Section 6).
Assessment	No live demo. Marks come from your submitted files, GitHub repository and video (Section 7).

1 Learning outcomes

- Assign clear responsibilities to objects and use encapsulation, inheritance, polymorphism and composition appropriately.
- Explain how the design changed across three working stages using UML, code and GitHub evidence.
- Implement a complete game with meaningful player choices, a clear result and robust input handling.

2 Game setup

2.1 Base setting

Your game must always offer this base setting, so that a TA can reproduce your runs. Other difficulty settings are optional.

Setting	Required base value
Turns	18 turns maximum
Tables	2
Staff	Waiter × 1 (one task per turn) Chef × 1 (one preparation unit per turn) Cashier × 1 (one payment per turn)
Arrivals	One customer at the start of turns 1, 4, 7, 10, 13 and 16 (six in total)

Setting	Required base value
Objective	At the end of turn 18: **at least 4 paid customers** AND revenue **at least $45** AND average satisfaction of paid customers **at least 60**

Menu item	Price	Preparation units
Salad	$9.00	1
Burger	$15.00	1
Pasta	$18.00	2

2.2 General rules

- **Customers choose their own food.** The manager never chooses it. Use a documented deterministic rule or seeded randomness, and show the choice once the order is placed.
- **Keep rules out of the UI.** The UI only collects the manager's decisions and displays the state. Game rules belong in domain and control objects.
- **Money.** If fractional prices arise, store amounts as integer cents. Display dollars with two decimals.
- **Offline.** All mandatory features must work without network access.

3 How a turn works

3.1 Start-of-turn display

At the start of every turn, show: the turn number, revenue, average satisfaction of paid customers, the waiting queue, occupied tables, each customer's satisfaction, order status, and the staff tasks available.

3.2 Turn steps

Each turn then runs these six steps in order.

Step	What happens
1 Arrivals	The scheduled customer joins the waiting queue with satisfaction 100.
2 Manager choices	**Waiter:** choose at most one task, or Wait: seat one queued customer at a free table, take one seated customer's order, or serve one READY order. **Chef:** choose at most one order to advance by one preparation unit, or Wait. The Chef may work on an order the Waiter took this turn.
3 Kitchen and serving	The Waiter acts before the Chef. When an order's last preparation unit is done, it becomes READY. A dish can be served only if it was already READY when the Waiter acts, so a dish finished this turn is served next turn at the earliest. Serving sets it to SERVED.

Step	What happens
4 Eating and payment	A customer served on an earlier turn finishes one full turn of eating and becomes READY_TO_PAY. The Cashier automatically handles at most one READY_TO_PAY customer per turn, oldest first. A customer who became READY_TO_PAY this turn pays next turn at the earliest. Payment adds the bill to revenue, marks the customer and order PAID, and frees the table.
5 Waiting and abandonment	At turn end, every waiting or seated customer who has not been served loses satisfaction (Regular: 8 points, minimum 0). A customer who reaches 0 leaves without paying: cancel any unfinished order and free the table. Customers who are served, eating or ready to pay lose nothing.
6 Result	The game always runs all 18 turns; there is no early ending. After turn 18: **Victory** if all three targets in Section 2.1 are met, otherwise **Defeat**.

3.3 Example: one customer from arrival to payment

This example follows one Regular customer who orders Pasta (2 preparation units) and gets the manager's full attention.

Turn	What happens	Satisfaction at turn end
1	Arrives (100). Waiter seats the customer.	92 (seated, not served)
2	Waiter takes the Pasta order. Chef does unit 1 of 2.	84
3	Chef does unit 2 of 2: order is READY. The Waiter cannot serve it this turn.	76
4	Waiter serves: SERVED.	76 (no further loss)
5	Customer finishes eating: READY_TO_PAY.	76
6	Cashier collects $18.00: PAID, table freed. Satisfaction 76 is recorded.	—

3.4 Rules that always apply

- A Waiter completes at most one task per turn.
- A table stays occupied until its customer pays or leaves.
- A customer cannot eat before being served. The Cashier cannot charge before READY_TO_PAY.
- An invalid choice must explain why it failed and let the manager choose again, without using up the staff task.
- Revenue never includes unpaid or abandoned orders.
- The customer who arrives on turn 16 cannot finish paying within 18 turns, so at most five customers can pay. This is intended, not a bug.

3.5 End-of-turn results

Show the served count, unhappy departures, revenue, average satisfaction of paid customers, and turns used. The average uses the satisfaction recorded when each customer pays. If no one has paid yet, show N/A.

4 The three stages

Each stage is a separate runnable version tagged in GitHub, and each builds on the one before.

Stage	Adds	Git tag
1 Basic operation	Regular customers, menu, Waiter, Chef, Cashier, full turn sequence, both endings	`stage-1`
2 Customer behaviour	VIP and Critic customers	`stage-2`
3 New role and menu policy	Host, ComboMeal, Happy Hour	`stage-3`

4.1 Stage 1: Basic restaurant operation

Build a complete playable game containing:

- regular customers and the three menu items;
- Waiter, Chef, Cashier, Table, Order and the waiting queue;
- the full turn sequence, including satisfaction, abandonment, payment, victory and defeat.

The player sets seating, ordering and serving priority through the Waiter, and kitchen priority through the Chef.

> **Stage 1 must:** compile and run on its own at tag `stage-1`; contain no Stage 2 or Stage 3 features; include one reproducible victory run and one reproducible defeat run in your test notes.

4.2 Stage 2: Customer behaviour extension

Add VIPCustomer and CriticCustomer alongside regular customers.

Customer	Satisfaction Loss per waiting turn	Bill	Special behaviour
Regular	8	Full price	—
VIP	5	10% off	—
Critic	12	Full price	A distinct order preference or reaction to service that you define and document

- A reproducible base-setting run must include at least one VIP and one Critic. You may replace two of the regular arrivals; there must still be six arrivals.
- All customer types follow the same order and service flow.
- Different behaviour must be dispatched through the customer objects, not through a chain of type checks in the turn controller.
- Stage 2 is independently playable and uses the same win and lose criteria.

4.3 Stage 3: New role and menu policy

Host. The Host seats at most one queued customer per turn, in addition to the Waiter's task. The Waiter can still seat customers.

ComboMeal. A combo is made of two existing menu items.

- Preparation units = the sum of its two components.
- Price = 90% of the components' combined price, rounded to the nearest cent. Example: Salad + Pasta = ($9 + $18) × 0.9 = $24.30, 3 units.
- A customer chooses a combo according to a rule you document.

Happy Hour. A pricing policy the manager may switch on.

- Can be activated once per game, at the start of a turn. It lasts for that turn and the next. Display its remaining duration.
- Orders taken while it is active get 20% off their bill.
- During each active turn, customers still waiting for a table or food recover 15 satisfaction at turn start (maximum 100).
- Apply the VIP discount first, then Happy Hour. The price is locked when the order is taken. Example: a VIP orders a Burger during Happy Hour: $15 × 0.9 × 0.8 = $10.80.
- It does not change orders already placed, and does not affect customers who have left.

Stage 3 is independently playable.

4.4 Design freedom and changes to earlier code

- You choose class names and design patterns. Do not create interfaces just to satisfy a checklist.
- If a later stage needs you to modify an earlier class, explain why. Small registration or setup changes are expected.
- If you broadly rewrite stable game rules, explain why and show that earlier behaviour still works.
- Extra features earn credit only when they show sound design and do not replace required rules.

5 Design evidence

Include these in your PDF report and repository.

Evidence	Requirements
Class diagram	Final design; must match the Stage 3 code. Show the principal entity classes, necessary control and boundary classes, relevant enums, key attributes and operations with visibility, inheritance, and associations with multiplicities where relevant. Annotate meaningful examples of encapsulation, polymorphism and composition.
Sequence diagram	One complete scenario in which a manager choice changes an outcome. Include: customer arrival and order; Waiter and Chef assignments; READY then SERVED; eating; payment; the result check; one Stage 2 or 3 behaviour (e.g. VIP pricing or Happy Hour). Show messages among the CLI, controller, staff and domain objects, with relevant guards and loops.

Evidence	Requirements
Design discussion	A table of key abstractions and their responsibilities. SRP, OCP, LSP, ISP and DIP, only where genuinely reflected in your code. Trade-offs, and one alternative design you considered. How the player's choices change a result.
GitHub evidence	Tags `stage-1`, `stage-2` and `stage-3` on runnable commits. In the README, for each transition (1 → 2 and 2 → 3): new files; existing files changed or deleted, with reasons; the relevant commit or compare link. The source code itself, not only the README, must show the real stages.

6 Submission

6.1 What to submit

Submit one archive per group through the course assignment link by the deadline announced on NTULearn (the earlier combat assignment's deadline does not apply). Name it `LABGroup_AssignmentGrp.zip`, for example `FCS1_Grp1.zip`.

Item	Contents
Source code	Java source with build and run instructions
README	Repository URL, stage comparisons (Section 5) and video timestamps
PDF report	Final diagrams, design discussion, functional evidence and reflections
Video	The recorded video, or an accessible download link if the file is too large
[redacted]	[redacted]

6.2 Recorded video

There is no live demonstration. Record one group video of about 15 minutes with readable screen capture and audible explanation. Each member explains a part they understand. List the timestamps of these segments in your README:

#	Segment
1	The game launching from the submitted project
2	Meaningful manager choices, including a decision where staff are constrained
3	A dish going READY → SERVED → payment
4	A VIP or Critic behaviour
5	A Stage 3 feature
6	One victory and one defeat (separate reproducible runs, or an accelerated replay)
7	The stage tag comparisons and the diagrams

The video is evidence for the marking criteria, not a separate score. If you submit a link, it must stay available to graders.

7 Assessment

Criterion	Marks	What it covers
UML class diagram	25	Coverage; relationships and notation; attributes, operations and encapsulation; clarity and match to code
UML sequence diagram	20	Scenario and objects; food and payment flow; OO messages and control notation; clarity
Design consideration	15	Responsibilities and cohesion; extensibility and principles; trade-offs and alternatives
Implementation code	20	Stage 1 core game; Stage 2 and 3 features; code quality; reproducibility
Thinking and development process	20	Stage snapshots; change analysis; report and reflection; recorded video
Total	**100**	